

OpenCollab + GLM-5.2

SWE-bench Lite 本地评测与 Team 模式实验总结

实验整理

2026-06-29

摘要

本报告汇总了 OpenCollab 接入 GLM-5.2 后，在 SWE-bench Lite 前 100 题上的一系列本地实验，并纳入后续 Team 模式、SWE 委员会 V1/V2 架构、空 patch 复跑、temperature=0.2、temperature=0.0、1M 预算补测、更新仓库补测、四题过题原因复盘与外部方法对照结果。

1 实验对象与口径

实验对象是 SWE-bench Lite 的前 100 个实例。patch 全部由本地 OpenCollab 仓库配合 GLM-5.2 生成，没有用任何线上服务。是否通过一律以 SWE-bench harness 给出的 `report.json` 为准，标准就是 `resolved=true`。

2 单 Agent 前 100 题结果

单 Agent 前 100 题最终可审计结果为 56/100。初始非空 patch 组共有 81 题，官方评测通过 50 题、未通过 31 题；初始空 patch 组共有 19 题，经补充评测后通过 6 题、未通过 13 题。两组合并后，单 Agent 在前 100 题上的最终状态是 56 通过、44 未通过。

阶段	样本数	通过	未通过	通过率
初始非空 patch	81	50	31	61.7%
初始空 patch 后续补齐	19	6	13	31.6%
单 Agent 当前可审计状态	100	56	44	56.0%

表 1: 单 Agent 前 100 题当前官方判定

值得一提的是，31 道非空但未通过的题里，patch 本身都能干净地 apply，失败来自目标测试仍未完全通过或少量 P2P 回归。初始空 patch 组在补充评测后也形成了完整官方判定。这个结果说明，前 100 题的主要瓶颈已经从 patch 格式转向语义修复质量、编辑策略和步数控制。

3 空 patch 研究

按旧记录，这 19 个空 patch 大致能分成三类。一类是曾有编辑动作但最终 diff 归零，一类是在编辑过程中停滞或撤回，还有一类从头到尾没有形成有效写入。它们的共同困难是最终 patch 字符串为空，单看结果无法区分模型未行动、行动后撤回、步数不足，还是工具状态中断。为此，后续分析把文件写入结果、最终 diff、循环检测和工具错误信号作为判别依据。

旧空 patch 类别	数量	12 步复跑后非空
曾改动但最终 diff 为空	5	5
改动后卡住或撤回	6	6
无有效写入	8	4

表 2: 初始空 patch 的旧分类与复跑结果

12 步复查后仍异常的样本提供了关键证据。django__django-11797 和 django__django-15738 更像是步数不足：会话已经接近定位根因，但还没有进入有效编辑。django__django-12589 反复读取同一大文件并触发 loop detector，说明部分空 patch 来自定位阶段原地打转。django__django-14238 则属于环境无效样本，经排除后纳入后续有效评测。

复查结论很清楚：步数上限过短会把“已经接近根因但尚未编辑”的会话截成空 patch。把上限放宽到 24 步后，django__django-11797、django__django-12589 和 django__django-15738 都形成了非空 patch，尽管官方评测仍未通过。因此，空 patch 不应被简单解释成模型完全没有能力；它至少混合了步数预算、定位停滞、写入可靠性和少量环境有效性问题。

4 Team 模式结果

组合策略	通过题数	通过率
单 Agent	56/100	56.0%
Team	78/100	78.0%

表 3: 单 Agent 与 Team 模式通过率对比

5 SWE 委员会模式架构

在原始团队模式之外，本实验后续设计了两代面向 SWE-bench 的显式工作流，分别记作 SWE 委员会 V1 和 SWE 委员会 V2。二者都遵守“盲验证”口径：生成补丁时只能使用问题文本、仓库源码、公开测试和公开文档，不能读取官方隐藏测试补丁，也不能读取官方注入的“失败转通过”测试节点或任何会泄露评分答案的信息。最朴素地说，委员会模式想解决一个问题：模型写代码前要先弄清“什么行为有证据、什么行为只是猜的”，写完代码后还要有人专门找边界、找回归、找没有证据支撑的改动。

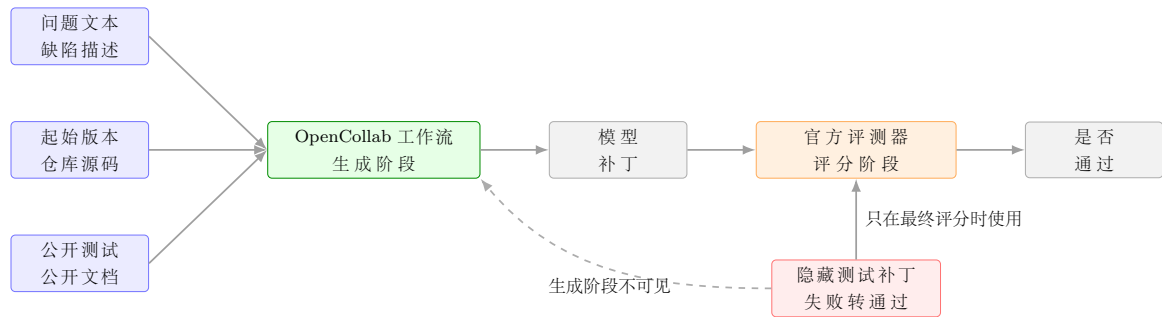


图 1: 从 SWE-bench 任务到模型补丁的盲验证闭环

上图先把输入边界讲清楚。OpenCollab 生成补丁时只能看到问题文本、起始版本源码、公开测试和公开文档。官方隐藏测试补丁和“失败转通过”信息只在最后的官方评测阶段使用，生成阶段的模型角色拿不到这些答案。因此，报告里说的“验证更强”，指的是工作流自己从公开信息中构造更好的验证，并没有提前读取官方评分测试。

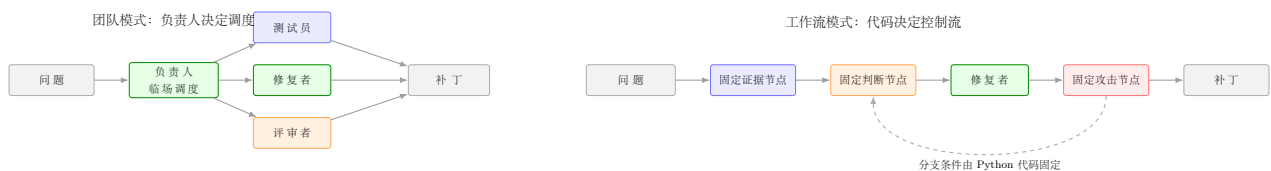


图 2: 团队模式与 Python 工作流的控制权差异

团队模式的不确定性主要来自负责人的临场调度：它可以决定什么时候派测试员、什么时候让修复者继续、什么时候停止。工作流模式保留每个节点内部的模型推理，但节点顺序、最大重试次数和分支条件由 Python 代码固定。换句话说，同样是多角色协作，团队模式更像自由讨论，工作流模式更像一套有岗位说明和放行规则的实验协议。

术语	本报告中的含义
任务实例	一个 SWE-bench 软件缺陷修复任务，通常包含问题文本、目标仓库、起始版本和官方评分测试。
起始版本	模型开始修复时看到的原始代码版本。所有补丁都在这个版本上生成。
模型补丁	工作流最终交给官方评测器的差异文件。
官方评测器	SWE-bench 官方评测程序，负责把补丁应用到仓库并运行隐藏测试，最后给出是否通过。
失败转通过测试	官方期望补丁后从失败变为通过的测试集合；盲验证下生成阶段不可见。
回归保护测试	官方期望补丁后继续通过的测试集合，用来检查修复有没有破坏原有行为。
临时验证	工作流自己从公开信息构造的临时检查，用来帮助修复者逼近隐藏行为。
行为契约	从问题文本、源码、公开测试或文档中抽出的行为约束，例如返回值、异常、排序、警告文本。
诊断信号	证据不足以写成硬断言，但能帮助定位风险或启发修复者的弱信号。
可见行为	外部用户或测试能观察到的行为，包括返回值、异常类型、错误消息、默认值、顺序和副作用。
小修重试	在已有证据和失败反馈下做小范围修补，避免重开完整探索。

这件事可以用一个很简单的心智模型理解。普通团队模式更像一群人围着一个缺陷商量，最后让修复者改代码；委员会工作流先把“证据”“测试”“写代码”“攻击补丁”“最终放行”拆成固定岗位。每个岗位只做自己的事，下一步能不能走，由结构化状态决定。这样做的目的，是减少两类常见失败：一类是修复者凭直觉写了一个看似合理、但隐藏测试不接受的边界；另一类是测试角色把没有证据的猜测当成预期行为，反而把修复者带偏。

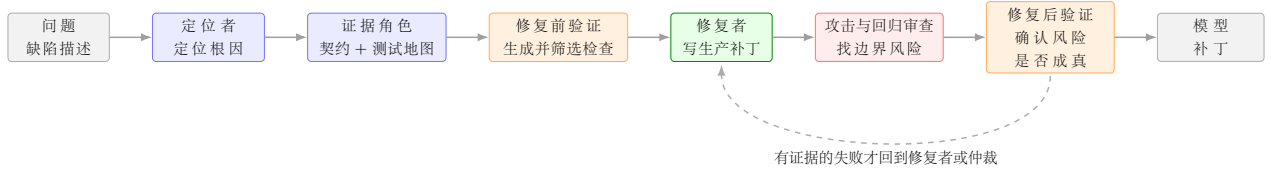


图 3: SWE 委员会模式的共同直觉：先建证据，再写补丁，再攻击补丁，最后放行

这张共同直觉图从左到右阅读。问题文本只是原始材料，它本身经常含糊、缺测试、甚至只描述一个用户症状；定位者负责把它变成可定位的根因假设。证据角色不写代码，只回答“我们到底知道哪些行为应该成立”。修复前验证把这些行为变成可运行或可诊断的临时检查。修复者只在这些证据约束下写生产补丁。写完以后，后半段角色关心补丁有没有改坏别的东西、有没有绕过某个边界、有没有把猜测写成事实。委员会模式的核心是固定模型角色的发言顺序和职责边界，多角色数量本身并非关键。

5.1 委员会 V1：验证委员会

委员会 V1 对应代码中的 `validation-council-solve`。后文以 SWE 委员会模式命名的剩余 13 题补测脚本，实际传入的也是这个 workflow。它是第一版结构化验证 workflow，重点是把“写代码”和“验证代码”分开。V1 先由定位者定位问题，再并行调用契约挖掘者和测试地图员。前者抽取行为契约，后者整理测试框架、测试夹具、相邻公开测试和临时验证建议。随后，修复前验证工厂根据契约提出候选验证，修复前验证裁判只保留少量有证据支撑的临时检查，起始版分诊器在起始版本上验证这些检查的可用性。进入修复阶段后，修复者生成最小生产补丁，补丁验证者检查补丁边界，补丁风险审计者依据差异寻找边界和回归风险，修复后验证工厂与修复后验证裁判再做一次补充验证，最后由最终验证者判断是否可以产出补丁。正文后续都使用中文岗位名；代码里的具体节点名保留在实现文件和测试文件中。

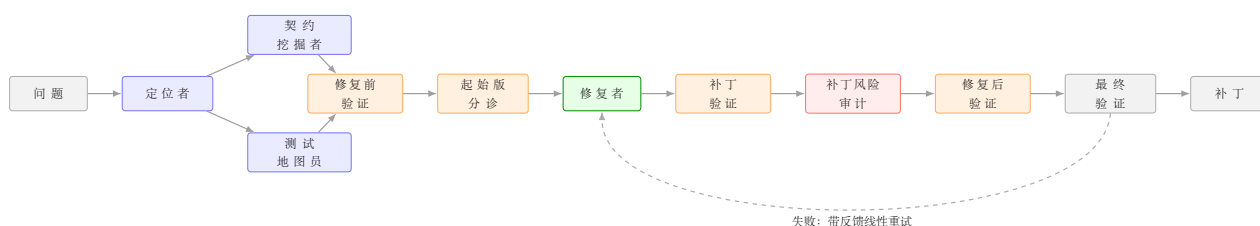


图 4：委员会 V1：先抽契约和测试地图，再让修复者在验证反馈下线性重试

V1 的图可以按两层理解。上半段是“修之前先确认目标”：契约挖掘者告诉系统要修的行为是什么，测试地图员告诉系统怎么在这个仓库里观察行为，修复前验证裁判决定哪些验证值得信。下半段是“修之后再审查”：补丁验证者看差异边界，补丁风险审计者找可能漏掉的边界情况，修复后验证判断这些风险是否需要反馈给修复者。V1 的重试是线性的，最终验证者如果看到失败，就把失败反馈传给下一轮修复者；如果看到阻断，就停止。这个版本的优势是成本较低、结构清楚，适合作为从普通团队模式到结构化验证的过渡。

5.2 委员会 V2：严格图同构的 SWE 委员会

委员会 V2 对应代码中的 `swe-committee-v2`。它保留 V1 的盲验证约束，但把图结构进一步细化成严格的证据、仲裁、攻击和最终审查四段。第一段是证据阶段：定位者之后并行运行契约挖掘者、测试地图员和可见行为清单员。新增的可见行为清单员专门列出返回值、异常类型、警告文本、错误码、默认值、顺序等外部可见字段，防止修复者在没有证据的字段上写硬断言。第二段是契约仲裁庭，它把三路证据合并为强证据契约、弱证据契约、猜测字段和禁止硬断言字段。只有强证据契约适合进入硬测试，弱证据更多作为诊断信号。

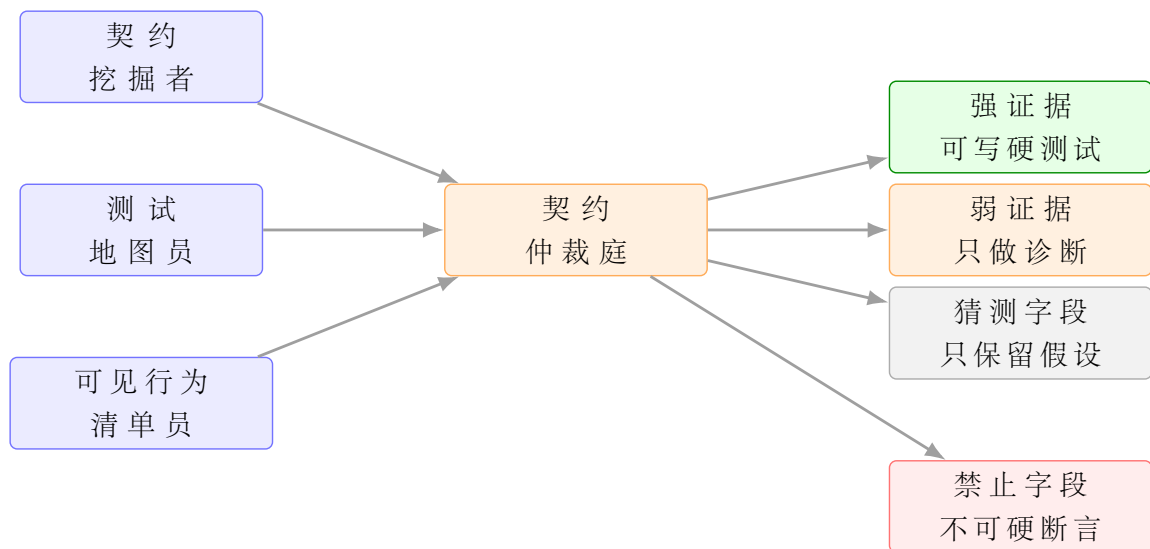


图 5: 委员会 V2 的证据分层：强证据、弱证据、猜测字段和禁止硬断言字段

这张证据分层图解释了 V2 的核心约束。强证据契约可以进入硬测试，因为它有问题文本、源码、公开测试或文档的直接证据。弱证据契约可以提醒修复者或测试员注意某个风险，但更适合写成诊断检查。猜测字段只能作为待确认假设保留，不能出现在预期行为里。禁止字段表示证据明确不足，后面的最终怀疑者会专门检查补丁有没有偷偷依赖这些字段。

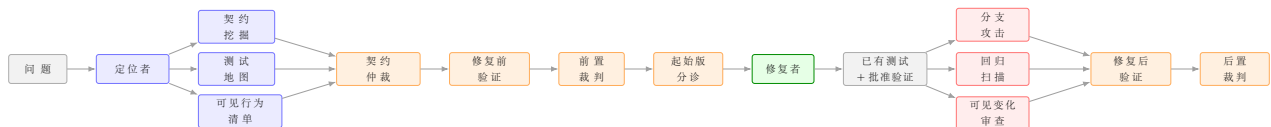


图 6: 委员会 V2 的证据与攻击阶段：三路证据先仲裁，补丁写完后三路攻击

这张 V2 主体图的左半边解决“证据从哪里来”：契约挖掘者负责行为契约，测试地图员负责仓库测试地形，可见行为清单员负责外部可见字段。三者都进入契约仲裁庭，由它决定哪些是强证据、哪些只是弱证据、哪些字段禁止硬断言。中间部分解决“什么时候可以写代码”：修复前验证和前置裁判只把证据足够的临时检查交给起始版分诊器，起始版本上的表现会告诉修复者哪些检查是有效复现、哪些只是诊断。右半边解决“补丁写完以后怎么被攻击”：已有测试和批准验证先检查当前补丁的基本状态，随后三路攻击角色分别从分支边界、邻近回归和可见行为变化三个角度找风险，再交给修复后验证和后置裁判。

V2 的后半段比 V1 更关键，因为它把“失败该回到哪里”写成了状态机。这里有三种失败信号。第一种是有证据失败，对应代码常量 `FAIL\_WITH\_EVIDENCE`，意思是修复后验证发现一个有证据支撑的检查在当前补丁上失败，这时应该直接交给修复者小修重试。第二种是契约证据阻断，对应 `CONTRACT\_EVIDENCE\_BLOCKER`，意思是最终怀疑者发现当前补丁依赖的行为契约本身证据不足，这时不能继续让修复者猜，应回到契约仲裁庭重新仲裁证据。第三种是具体问题阻断，对应 `CONCRETE\_BLOCKER`，意思是最终验证者发现了具体可操作的问题，比如测试没过、差异边界不干净、临时文件没清理，这时也交给修复者小修重试，但不需要重开契约仲裁。

角色	权限	输入	输出
可见变化审查者	只读	当前差异、可见字段清单	未经证据授权的可见行为变化
最终怀疑者	只读	补丁、契约分层、修复后验证	证据不足阻断或通过
最终验证者	只读 + 可运行测试	补丁、所有审查结果	模型补丁放行或具体阻断

表 4: 委员会 V2 中各角色的职责、权限和产物

状态边	一个直观例子	为什么这样走
后置裁判到小修者	修复后检查发现公开证据支持的空字符串边界仍然失败。	失败已经足够具体，直接让修复者做最小修补。
最终怀疑者到仲裁庭	补丁改了异常类型，但问题文本、源码和公开测试都没有证据证明这个异常类型应当稳定。	问题出在契约证据层，继续让修复者改会变成猜测；应回到仲裁庭重分层。
最终验证者到小修者	最终检查发现差异里混入临时复现文件，或公开测试命令尚未通过。	这是具体可操作的提交质量问题，修复者删除临时文件或补跑验证即可。
最终验证者到产物	证据充分、修复后验证通过、最终怀疑者通过、最终差异也干净。	此时产物可以作为模型补丁交给 SWE-bench 官方评测器。

维度	委员会 V1	委员会 V2
前置证据	定位者之后并行抽取行为契约与测试地图。	定位者之后并行抽取行为契约、测试地图与可见行为清单。
契约处理	契约集合直接交给验证生成者与修复者。	契约仲裁庭把证据分成强证据、弱证据、猜测字段和禁止字段。
修复前验证	验证生成者、裁判、起始版分诊器三步串联。	同样保留，但硬预期只能来自强证据契约，弱证据只做诊断。
修复后审计	补丁验证者、补丁风险审计者、修复后验证、最终验证者。	已有测试和批准验证之后进入三路补丁攻击，再由后置裁判、最终怀疑者、最终验证者分流。
重试逻辑	最终验证者看到失败后线性重试，最多一轮修复者重试。	“有证据失败”“契约证据阻断”“具体问题阻断”三条边显式分流，最多两轮小修重试。
主要目的	让 SWE 解题过程可审计，并减少凭空测试和临时文件。	把“证据不足”和“具体修复错误”分开处理，强化边界测试、回归扫描和可见行为审查。

从科研角度看，V1 解决的是“让验证变成独立角色”的问题，V2 进一步解决的是“哪些失败应该

回到契约仲裁，哪些失败应该直接交给修复者修”的问题。这个区别很重要：如果把所有失败都交给修复者，模型容易在证据不足的假设上反复修补；如果把所有失败都回到仲裁，流程又会过重。V2 通过有证据失败、契约证据阻断和具体问题阻断三种结构化状态，把修复后验证、最终怀疑者和最终验证者的职责边界固定下来。后续若继续评测剩余失败题，V2 更适合用来研究“更强验证数据”和“多角色审查”是否能减少只差一两个边界情况的失败。

5.3 装傻评审追问与澄清

为了检查这段方法说明是否足够清楚，我们用一个完全不了解 OpenCollab 的读者视角重新追问。下面这些问题都很基础，但正是这类问题最容易暴露方法章节有没有把机制讲透。

装傻评审问题	报告中的澄清回答
为何要先建证据，再让修复者改代码？	SWE-bench 的问题文本往往只描述症状，隐藏测试检查的是更精确的行为契约。直接改代码容易把“猜测”当成“事实”。委员会模式先抽契约、测试地图和可见行为字段，让修复者在有证据的边界里动手。
V1 和普通团队模式到底有什么差别？	普通团队模式的分工更多依赖负责人临场组织。V1 把定位者、契约挖掘者、测试地图员、验证裁判、修复者、补丁风险审计者、最终验证者固定成 workflow 节点，并且限制每个节点的工具和输出。差别在于控制流由代码决定，负责人不能随意跳过验证。
V2 为什么要多一个可见行为清单员？	很多失败来自外部可见行为没想全，例如异常类型、警告文本、排序、默认值和返回形状。可见行为清单员专门把这些字段列出来，后面的可见变化审查者才能判断补丁是否改了没有证据支撑的行为。
契约仲裁庭和普通评审者有什么差别？	它更像证据仲裁器。它的职责是契约分层：强证据可以写硬测试，弱证据只能诊断，猜测字段不能写成预期行为，禁止字段不能被补丁偷偷改变。
为什么后置裁判失败后不去最终验证者？	后置裁判的失败已经是“有证据的补丁后检查失败”。这时问题足够具体，继续交给最终验证者只会重复审查，直接让修复者做最小修补更合适。
为什么最终怀疑者有时回契约仲裁庭？	最终怀疑者处理的是证据不足。证据不足时，修复者再改也只是在猜；正确动作是回到契约仲裁庭，重新降低或修正契约，再重新生成修复前验证。
最终验证者回小修和后置裁判回小修有什么区别？	后置裁判回小修来自有证据临时检查的失败，说明某个行为没修好。最终验证者回小修来自最终可提交性问题，例如差异带了临时文件、测试边界不干净、补丁路径不合规。二者都回修复者，但反馈内容不同。
这套结构为什么可能帮助那种只差一两个边界情况的问题？	这类失败通常说明主方向已经接近正确，只缺少某个边界验证。V2 在写补丁前强制抽契约，在写完后做分支攻击、回归扫描和可见变化审查，就是为了系统性制造这些边界验证。

表 5: 装傻评审追问暴露出的关键澄清点

最后需要说明这一节和前文数字之间的关系。前文的单智能体 56/100 与团队模式 78/100 是已经完成的本地评测结果；更新仓库补测把前 100 题累计提高到 87/100，使用的是当时更新后的团队模式和评测配置。随后剩余 13 题的 SWE 委员会补测使用的是 V1，也就是 `validation-council-solve`。V2 是在 V1 经验上继续细化出的图同构工作流，应单独作为新的委员会实验报告，和前面的团队模式及 V1 数字分开统计。

6 Team 模式失败案例分析

Team 模式没解出来的题，patch 同样都能正常 apply，卡点全在测试。具体看，`django__django-13590` 和 `django__django-13660` 是另一种形态——目标测试过了，却把原本通过的 `PASS_TO_PASS` 改回归了。其余大多数样本则是 `FAIL_TO_PASS` 还有残留。换句话说，到这一步，Team 模式的失败已经和 patch 能不能用没关系了，纯粹是语义层面的修复还不够准。

实例	F2P 失败	P2P 失败	首个失败信号
<code>astropy__astropy-14365</code>	1	0	QDP roundtrip case handling
<code>astropy__astropy-7746</code>	1	0	zero-size WCS input
<code>django__django-13321</code>	18	0	invalid session data decoding
<code>django__django-11019</code>	13	0	forms media combine/construction
<code>django__django-11283</code>	1	0	auth proxy permission migration
<code>django__django-11564</code>	2	0	SCRIPT_NAME static/media prefix
<code>django__django-11630</code>	2	0	duplicate db table checks
<code>django__django-11742</code>	2	0	choice max length checks
<code>django__django-11848</code>	2	0	RFC 850 date parsing
<code>django__django-11905</code>	2	0	isnull lookup behavior
<code>django__django-12470</code>	1	0	inherited ordering by pk desc
<code>django__django-13590</code>	0	5	iterable lookup expression regressions
<code>django__django-13660</code>	0	2	shell command globals regressions
<code>django__django-13768</code>	1	0	send_robust logging behavior
<code>django__django-14017</code>	1	0	Q object and boolean expression combination
<code>django__django-14155</code>	3	0	ResolverMatch repr for partial functions
<code>django__django-14730</code>	1	0	related_name on symmetrical many-to-many
<code>django__django-15202</code>	2	0	URLField validation
<code>django__django-15213</code>	1	0	full expression annotation aggregation
<code>django__django-15252</code>	2	0	TEST MIGRATE=False schema behavior
<code>django__django-15695</code>	1	0	unnamed index rename migration

实例	F2P 失败	P2P 失败	首个失败信号
django__django-15738	1	0	FK to M2M migration ordering

7 失败模式归因

为了搞清楚这 22 题到底卡在哪,我们把两批官方评测 (Team 34 题里的 19 个未解 + Team 10 题里的 3 个未解,正好凑成 22) 的 `patch.diff` 和 `test_output.txt` 都翻了一遍,并抽查了覆盖各种失败规模的样本——大面积挂的 `django__django-11019` 和 `django__django-13321`、引入 P2P 回归的 `django__django-13590` 和 `django__django-13660`、只差一两个用例的 `django__django-14155` 和 `django__django-14017`,以及两道 `astropy`。结论相当集中。这 22 题没有一道是 `patch` 用不了的,全部成功 `apply`。失败几乎都是同一回事,模型找对了文件、也改在了正确的函数上,但语义修复差了一口气。换句话说,到这一步工程和格式已经退居次要位置,核心问题变成了模型对 bug 根因的理解还不够精确。

具体可以拆成三类。第一类是**改对了位置、思路也接近,但实现写错了**。`django__django-13590` 把 `type(value)(...)` 改成 `type(value)(*[...])`,可 `tuple()/list()` 只收一个参数,直接报 `TypeError`,还把原本通过的 5 个 P2P 测试带崩了。`django__django-14017` 给 `Q._combine` 加了 `Q(other)` 包装,但方式不对,跑出 `'Exists' object is not subscriptable`。`django__django-11019` 把 `media merge` 整个重写成拓扑排序,方向其实是对的,但依赖图实现有 bug,排序结果错,13 个 F2P 全挂。`django__django-13321` 在 `_legacy_decode` 里只 `catch` 了 `binascii.Error`,改动范围太窄,和正解的整体重构对不上,18 个 `session` 测试挂掉。

第二类是**只实现了一半,漏改了配套的另一处**。`django__django-14155` 加了 `functools.partial` 的解包逻辑,却没同步改 `__repr__`,测试期望的带引号 `url_name` 显示没出来。`django__django-13660` 知道 `exec` 要传命名空间,把 `exec(cmd)` 改成了 `exec(cmd, {})`,但传的是空 `dict`,丢了 `globals`,命令里的 `import` 全失效,断言 `'False' != 'True'`。`astropy__astropy-14365` 给读取正则加了 `re.IGNORECASE` (这正是正解),可只修了读的路径、没修写出路径,于是 `roundtrip` 的大小写对不上,还顺手在仓库根目录留了个垃圾 `test.qdp`。`astropy__astropy-7746` 加了零尺寸输入的处理,但边界返回的形状不全,差一个 F2P。

第三类数量最多,是**差一两个边界用例的近失**。`django__django-11283`、`-11564`、`-11630`、`-11742`、`-11848`、`-11905`、`-12470`、`-13768`、`-14730`、`-15202`、`-15213`、`-15252`、`-15695`、`-15738` 都属于这一档——F2P 只挂 1 到 3 个、P2P 全过,`patch` 通常也就十几到三十行,主逻辑改对了,只是某个特定 `case` (空值、命名约定、迁移顺序之类) 没覆盖到。

失败模式	题目数	代表样本
实现写错（思路接近）	4	django__django-13590、-14017、-11019、-13321
只改了一半	4	django__django-14155、-13660、astropy__astropy-14365、astropy__astropy-7746
差边界用例的近失	14	django__django-11742、-11848、-15695 等

有两点值得单独标注。其一，真正引入 P2P 回归的只有 django__django-13590 和 django__django-13660 两题，也就是说”改对了目标却把别处弄坏”的情况很少，绝大多数失败是目标测试本身还没全过——这说明当前 GLM-5.2 整体偏保守，乱改情况较少。其二，astropy__astropy-14365 的测试日志里混着 numpy C 扩展的编译报错（expected PyArrayObject*），而它的修复思路其实就是正解，所以这道题的失败到底是逻辑差一截（没改写出路径）还是扩展重编译的环境噪声，值得单独重跑确认，不宜直接算成纯模型失败。

8 重复行为与温度设置

实验配置记录显示，早期 Team 运行使用 temperature=1.0，并关闭 thinking。这个设置沿用了仓库里为避免低温重复而设置的默认覆盖逻辑，也构成后续 temperature=0.2 和 temperature=0.0 对照实验的基线。

我们对 sessions/*/agent_*.json 做了去空白后的精确比对，凡是长度超过 80 字符的 assistant 输出，没有一条是完全重复的。Kimi 旧日志里那些 PROSE-STOP、unknown_tool、loop_blocked 信号，这次也一个没见到。

工具层面倒是有一些重复，但都很局部。Team 34 的事件日志里一共 15 条 loop_detected，散在 7 个文件中，基本都是同一个文件被 file_read 读了 8 遍以上，典型的有 django__django-13265、django__django-14730、django__django-15738 和 django__django-13590。真正的精确重复调用只集中在两处。django__django-13590 的 tester 把 django/db/models/sql/query.py 同一个 22 行窗口读了 5 次，django__django-14667 的 tester 把同一个目标测试跑了 3 遍。空 patch 复跑里也有类似苗头——django__django-11019 命中过重复 file_write，django__django-12589 和 django__django-15738 命中过重复读同一文件。总的看，这批 GLM-5.2 有少量工具重复和局部原地打转，但没有出现 Kimi 旧实验那种长时间复述同一段输出的硬死循环。

9 低温对照实验（temperature=0.2）

前面的运行都用 temperature=1.0。为了观察降温能否提升那些”思路对、实现差一口气”样本的通过率，随机挑了四道题做试验。

题目	官方结果	主要情况
astropy__astropy-7746	未通过	仍失败 <code>test_zero_size_input</code>
django__django-11019	未通过	14 个 F2P 失败，生成阶段出现 <code>loop_detected</code> 和 <code>budget_exceeded</code>
django__django-13321	未通过	18 个 legacy decode 相关 F2P 仍失败
django__django-13590	通过	修掉了 <code>namedtuple range lookup</code> ，且没有 P2P 回归

表 7: 低温 temperature=0.2 前 4 题官方评测

真正有价值的是 `django__django-13590`。`temperature=1.0` 的 Team 版当时是目标测试过了、却带了 P2P 回归（见前文失败模式归因），降到 0.2 之后这次官方评测直接通过了。另外三题未因低温获得提升。从过程看，0.2 确实更保守，但并不一定更快——`django__django-11019` 反而出现了局部反复加预算耗尽。

10 低温对照实验 (temperature=0.0)

最后一组实验把 Team 模式的温度降到 0.0，对象是 Team temperature=1.0 仍未解的 22 题。生成阶段 22 题都留下了 metrics，成功记录合计 6,535,787 tokens。

过程监测显示，22 题里有 14 题没有明显重复信号，4 题出现 `budget_exceeded`，4 题出现 `loop_detected` 或局部重复工具调用。所有 patch 都能 apply，官方评测最终通过 5 题、未通过 17 题，主要失败仍来自 F2P 残留。这个结果说明，`temperature=0.0` 能带来更强确定性，但也更容易放大局部重复和预算耗尽风险。

实例	token	步数	patch	生成期信号
astropy__astropy-14365	61,237	5	617	无明显重复信号
astropy__astropy-7746	136,083	7	1,106	无明显重复信号
django__django-13321	176,933	8	720	无明显重复信号
django__django-11019	514,364	12	5,347	<code>budget_exceeded</code>
django__django-11283	270,749	13	921	<code>run_tests</code> 重复
django__django-11564	272,169	17	727	无明显重复信号
django__django-11630	503,970	19	1,885	无明显重复信号
django__django-11742	305,510	19	1,578	无明显重复信号
django__django-11848	104,304	8	920	无明显重复信号
django__django-11905	197,149	8	938	无明显重复信号
django__django-12470	194,617	22	816	无明显重复信号
django__django-13590	372,419	6	941	<code>bash</code> 重复
django__django-13660	237,894	9	1,600	<code>run_tests</code> 重复
django__django-13768	93,376	11	995	无明显重复信号
django__django-14017	508,882	8	1,267	<code>budget_exceeded</code>
django__django-14155	228,686	12	783	无明显重复信号

实例	token	步数	patch	生成期信号
django__django-14730	260,514	13	1,036	无明显重复信号
django__django-15202	187,143	5	1,460	无明显重复信号
django__django-15213	501,525	16	2,042	budget_exceeded
django__django-15252	508,354	20	2,064	budget_exceeded
django__django-15695	395,795	13	1,287	无明显重复信号
django__django-15738	504,114	19	2,367	file_read 重复; 官方评测未通过

表 8: temperature=0.0 的 22 题生成 token 与过程信号

实例	结果	F2P	P2P	首个失败信号
astropy__astropy-14365	未过	1	0	QDP roundtrip
astropy__astropy-7746	未过	1	0	zero-size WCS input
django__django-13321	未过	18	0	cookie session legacy decode
django__django-11019	未过	16	4	forms media combine/construction
django__django-11283	未过	1	0	proxy permission migration
django__django-11564	未过	2	0	SCRIPT_NAME static/media prefix
django__django-11630	未过	2	0	duplicate db table checks
django__django-11742	未过	2	0	field choices max length checks
django__django-11848	通过	0	0	–
django__django-11905	未过	2	0	isnull lookup behavior
django__django-12470	未过	1	0	inherited ordering by pk desc
django__django-13590	通过	0	0	–
django__django-13660	通过	0	0	–
django__django-13768	未过	1	0	send_robust failure logging
django__django-14017	通过	0	0	–
django__django-14155	未过	3	0	ResolverMatch repr
django__django-14730	未过	1	0	useless related_name on M2M
django__django-15202	未过	2	0	URLField validation
django__django-15213	通过	0	0	–
django__django-15252	未过	2	0	TEST MIGRATE=False schema behavior
django__django-15695	未过	1	0	unnamed index rename migration
django__django-15738	未过	2	0	FK to M2M migration ordering

表 9: temperature=0.0 的 22 题官方评测结果

11 重复句直到预算耗尽的取证

事件日志确认，temperature=0.0 实验中确实出现过“同一句判断反复输出，直到预算耗尽”的模式。最核心的个案是 `django__django-15738`: coder 连续 25 次重复“编辑已经成功应用”的核心判断，16 次重复把工具失败解释为环境问题，21 次重复表示要换一种方式验证。随后同一会话触发 coder 与 lead 的 `budget_exceeded`。这个样本说明，低温下的重复可能表现为稳定判断被反复确认，也可能表现为自然语言整段复读。

这个模式的关键不在于模型完全不知道自己在重复。它每次都认为“编辑已经成功，工具报错只是环境问题，所以再换一种方式验证”。问题是，每一次“换一种方式”实际仍然回到同一个验证动作附近，loop detector 也连续报出 `bash` 重复，计数从 3 增到 18。也就是说，这里出现的是**自我确认被工具失败放大**：模型把一次 `str_replace` 的成功信号当作锚点，随后把所有读文件和跑测试失败都解释成环境噪声，于是不断尝试证明同一个结论，最后耗光预算。

实例	温度	重复证据	后果
django__django-15738	0.0	edit-applied 核心句 25 次；工具失败解释 16 次；different-approach 计划 21 次；bash loop count 3-18。	重复判断与工具 loop 同时上升，最终触发预算耗尽；该轮 patch 未修改生产代码，官方评测未通过。
django__django-13590	0.0	list/tuple 句 17 次；handle-both 句 10 次；upstream-fix 句 10 次；stop-looping 自覚 3 次。	触发 9 条 loop detector，但没有预算耗尽；最终跳出循环并生成 patch。这个样本说明 loop detector 能发现问题，却没有强制改写策略。
django__django-11019	0.0	工具访问失败说法 3 次；retry 说法 6 次。	多个 agent 相继预算耗尽。这里以工具访问失败引发的反复重试为主，纯文本复读成分较少。

表 10: temperature=0.0 中与重复输出和预算耗尽相关的主要证据

这三个样本合在一起说明，temperature=0.0 确实更容易让 agent 固着在一个确定性解释上。它不一定表现为旧 Kimi 那种一整段自然语言无意义复述；更多时候是“同一个判断句 + 同一种工具尝试”循环出现。15738 是最严重版本，重复句、loop detector 和预算耗尽三者完全重合。13590 是中间状态，模型已经意识到自己在循环，但 loop detector 只记录信号，没有把它强制切换成新的行动策略。11019 则提醒我们，工具访问失败也会诱发相似的重试行为，不能把所有重复都归为模型语义能力问题。

对 harness 来说，单纯记录 loop_detected 还不够。更合适的处理是：当同一 agent 在短窗口内重复同一自然语言判断超过 3 次，同时工具 loop count 继续上升，就强制进入“停止验证、保存当前 diff、换角色复核”的分支。若重复句里含有“edit was already applied”“environment issue”“retry”等关键词，还应保存最近一次成功写入、当前 diff、最近三次工具错误和当前 patch。这样可以减少 15738 这种预算消耗，同时保留足够证据判断到底是工具层故障、模型误判，还是 patch 提取出了问题。

12 1M token 预算复跑

前面 temperature=0.0 的实验里，有 4 题触发 budget_exceeded。因此，我们对这 4 个预算耗尽样本设置 1M token 上限进行复测。

这次 4 题全部生成了非空 patch，官方 SWE-bench 评测通过 1 题、未通过 3 题，没有 patch apply failure。django__django-14017 在 1M 预算下通过，说明它此前确实存在预算不足或过早停止的影响。django__django-11019、django__django-15213 和 django__django-15252 继续未通过，且失败都来自 F2P 残留；这些样本加预算后仍没解决，说明瓶颈已经更接近语义修复本身。

实例	token	patch 字节	用时 (s)	结果	F2P/P2P	首个失败信号
django__django-11019	706,356	5,475	573	未过	14/0	forms media combine/construction
django__django-14017	441,387	1,224	496	通过	0/0	-

实例	token	patch 字节	用时 (s)	结果	F2P/P2P	首个失败信号
django__django-15213	1,014,086	4,630	802	未过	1/0	full expression annotation with aggregation
django__django-15252	531,489	915	477	未过	2/0	TEST MIGRATE=False schema behavior

表 11: temperature=0.0、1M 预算四题复跑

13 更新仓库补测结果

在上述 1M 预算复跑之后，前 100 题的可审计状态是 83/100。随后，我们在更新后的 OpenCollab 仓库上，用 Team 模式、temperature=0.0、thinking 关闭的配置，对剩余 17 个未通过样本重新生成 patch 并进行官方评测。最终 4 题通过、13 题仍未通过，前 100 题累计结果提升到 87/100。相比之前的旧版本，新增通过样本是 django__django-11742、django__django-12470、django__django-15202 和 django__django-15738。

这四题的共同点是，新 patch 都能 apply，且官方评测中 F2P/P2P 均清零。与旧失败 patch 对比后可以看到，新增通过不能归因于官方评测随机波动，核心变化在于生成出的 patch 发生了实质变化：新版运行更频繁地构造接近隐藏测试的局部验证，coder/tester 对 lead 方案的纠偏也更充分。django__django-15738 的通过 patch 只包含 django/db/migrations/autodetector.py 的生产代码改动，关键是让 AddField 在 m2m/concrete 同名替换分支里依赖同名 RemoveField，从而满足迁移操作顺序断言。

14 四题从未过到通过的原因分析

这一轮需要单独回答的问题是：django__django-11742、django__django-12470、django__django-15202 和 django__django-15738 为什么现在可以通过，而旧仓库运行时没有通过。先看官方评测口径本身：旧候选和新候选的 patch 都能 apply；旧候选失败都来自 FAIL_TO_PASS 残留，新候选全部变成 F2P/P2P 双零。因此，差异不能由官方评测随机翻转解释，主要来自新旧运行实际生成了不同 patch，或者旧运行虽然找到方向但没有把正确 diff 保存出来。

更关键的是，这里要判断的是 Harness 本身变厚以后有没有把模型从“会想但做不准”推到“能生成可过 patch”。从 git 历史看，新实验前上游合入了一组会直接改变 agent 行为的改动：per-call budget 与 steering budget、prompt 去重和压紧、enforcement_strength 程序、角色工具白名单、证据型角色突破报告长度限制、recon 发现传递给 coder、identifier 使用前验证，以及按角色持久化运行轨迹。这些改动会改变 lead 给 coder/tester 的任务粒度、证据传递、验证强度和跑偏时的约束。与此同时，四题补测也混有温度从 1.0 到 0.0 的变化，以及更宽的预算设置。因此，本节只能做基于日志的归因强弱判断，不能把一次 A/B 外观差异直接当成严格因果实验。

实例	旧 F2P 失败	新 F2P 失败	旧 patch 问题	新通过原因
django__django-117 42	2	0	另起 <code>_check_choices_max_length()</code> ，消息模板不符合隐藏测试，并且 <code>named group</code> 行为仍不对。	新 patch 把 E009 检查放进 <code>_check_choices()</code> 结构校验成功分支，使用 <code>flatchoices</code> 和官方消息模板； <code>tester</code> 明确复核 <code>named group</code> 、边界相等和 6 字符错误消息。
django__django-124 70	1	0	旧 patch 只把递归参数从 <code>order</code> 改成 <code>default_order</code> ，仍未解决 <code>inherited -pk shortcut</code> 的根因。	新 patch 采用更接近上游的 <code>guard</code> ：当字段名就是 <code>pk</code> 或 <code>attname</code> 时不递归到相关模型 <code>ordering</code> ； <code>coder</code> 曾反驳 <code>lead</code> 的旧方向，并用 <code>queries</code> 回归验证说明旧方向会伤到 <code>FK ordering</code> 。
django__django-152 02	2	0	旧 patch 已处理 <code>urlsplit()</code> 的 <code>ValueError</code> ，但 <code>hostname is None</code> 时选择跳过长度检查，漏掉隐藏测试里的空 <code>hostname</code> 。	新 patch 把 <code>hostname is None</code> 也判作非法 URL； <code>tester</code> 直接读取并运行 <code>test_urlfield_clean_invalid</code> 和 <code>test_urlfield_clean_not_required</code> ，还专门构造空 <code>hostname</code> 对抗样本。
django__django-157 38	1	0	旧版 <code>Team/Harness</code> 未触发预算上限，patch 能 <code>apply</code> ，但只在 <code>_generate_add_field()</code> 内泛化追加依赖，还带入 <code>repro.py</code> ，官方 <code>\#23938</code> 仍失败。	当前 patch 只改 <code>autodetector.py</code> ，让调用点传入依赖，并仅在 <code>m2m/concrete</code> 同名替换分支令 <code>AddField</code> 依赖同名 <code>RemoveField</code> ，满足操作顺序断言。

表 12: 四题新旧运行的官方结果和根因对比

实例	Harness 工艺介入证据	归因判断
django__django-117 42	新日志把上游错误消息、 <code>fields.E009</code> 、 <code>flatchoices</code> 、 <code>named group</code> 和边界样例直接传给 <code>coder</code> ； <code>tester</code> 又模拟 <code>test_choices_in_max_length</code> 并跑完整 <code>invalid_models_tests</code> 。	工艺作用较强。旧运行也找到方向，但任务拆分和验证目标不够贴近隐藏测试，最终消息模板和插入位置都偏了。
django__django-124 70	新 <code>lead</code> 起初仍提出旧方向， <code>coder</code> 用 <code>-note FK ordering</code> 回归风险反驳，并改用 <code>pk shortcut guard</code> ；运行轨迹明确记录 <code>coder pushback</code> 。	工艺作用中等偏强。真正关键是角色间纠偏和回归意识，但这题也可能受单次采样影响。
django__django-152 02	新 <code>analyst</code> 定位到 <code>regex</code> 成功分支里的两个未保护 <code>urlsplit()</code> ； <code>tester</code> 明确复核两个 <code>FAIL_TO_PASS</code> 、 <code>required=False</code> 、 <code>valid URL</code> 、 <code>hostname is None</code> 和 <code>validators</code> 套件。	工艺作用较强。旧 patch 已包住 <code>ValueError</code> ，但漏掉 <code>hostname is None</code> ；新验证把这个边界直接打穿。
django__django-157 38	旧非空失败 patch 已理解同名替换排序问题，但依赖加法太泛化。当前通过 patch 把依赖改成调用点传入，并只在 <code>m2m/concrete</code> 分支绑定 <code>AddField</code> 到同名 <code>RemoveField</code> 。	归因混合。更厚 <code>Harness</code> 和更长预算都可能做贡献；现有证据不能单独证明结构升级是主因。

表 13: 四题从失败到通过时的 Harness 工艺归因

逐题看，提升来源并不完全相同。django__django-11742 属于“方向对、细节错”的失败。旧日志里 tester 自己构造了行为样例并宣称 PASS，但官方隐藏补丁要求精确错误消息：'max_length' is too small to fit the longest value in 'choices' (%d characters).，同时要求 named group 只统计真实 choice value。旧 patch 的消息和插入位置都不够贴合官方断言。新运行的 analyst 先读了 E008 附近的错误编号和 _check_choices() 原始结构，coder 把检查放进结构校验成功后的 else 分支，tester 又跑了 invalid_models_tests 全套以及边界样例。这说明新版运行的验证更贴近隐藏测试，单纯的模型采样变化解释力有限。

django__django-12470 是最能说明交互改进的一题。旧 patch 的一行修改看起来很合理，本地日志也跑了 ordering 相关套件，但官方仍挂 test_inherited_ordering_pk_desc。新运行里，lead 起初也倾向于类似旧方向；后续 coder 用证据指出这个方向会破坏 -note 一类 FK ordering，于是改用 guard 方案，让 pk shortcut 不再错误递归到相关模型的 Meta.ordering。这更像是 coder/tester 对 lead 方案的有效纠偏。官方通过来自 compiler.py 的生产代码改动。

django__django-15202 的变化最小，也最能削弱“随机官方评测”解释。旧 patch 和新 patch 都处理了 malformed IPv6 触发的 ValueError，但旧 patch 对 hostname is None 使用 hostname is not None and len(hostname) > 253，这会让隐藏补丁中的 \#0A.b0 漏过去。新 patch 改为 hostname is None or len(hostname) > 253，正好覆盖空 hostname。这里的提升来自多一个边界条件，而这个边界条件在新 tester 日志中被明确点名并验证。

django__django-15738 更合适的比较对象，是旧版 Team/Harness 中那次非空但未过的 patch。旧版运行没有触发预算上限，官方显示 patch apply 成功，但 \#23938 仍未过。旧 patch 的核心策略是在 _generate_added_field() 里凡是发现 from_state 里已有同名字段，就追加 order_wrt_unset 和 foo_together_change 依赖。这个 guard 看起来保护了同名替换场景，却没有把新 AddField 明确排在同名 RemoveField 之后。当前通过 patch 的关键差异是把依赖从调用点传入：_generate_added_field() 接受可选 dependencies，只在 m2m/concrete 不能直接 alter 的分支里，把 AddField 绑定到同名 RemoveField 之后。这个改动直接对应隐藏测试要求的 AlterUniqueTogether, RemoveField, AddField 顺序。因此 15738 的提升主要来自修复策略更精确；它不能证明所有提升都来自 harness 结构，也不能归为官方随机波动。

总体判断是：官方 SWE-bench 对固定 patch 的判定基本确定，本轮四题通过主要来自生成出来的 patch 与旧 patch 不同。这里确实可能有模型采样随机性参与，因为低温也不能保证每次走同一条推理路径；但更强的解释是新版运行在验证层面更接近隐藏测试，并且在 11742、12470、15202 这三题上都能看到任务分解、证据传递、对抗验证或角色纠偏的直接日志证据。换句话说，Harness 变厚很可能是前三题提升的重要原因；15738 的证据更混合，预算加长也可能影响了最终结果。后续若要严谨量化“harness 变强”本身，需要固定同一模型、同一温度、同一题集，在旧 harness 和新 harness 上各跑多次，再比较非空 patch 率、F2P 清零率和 tester 是否命中隐藏行为。

15 更新后仍失败的十三题

更新仓库补测后仍有 13 题失败。它们有一个共同点：patch 全部能 apply，P2P 回归全为 0，也没有官方 harness error 或空 patch。失败全部来自 FAIL_TO_PASS 残留，说明问题已经从“产物拿不到/patch 应用失败”转为“语义修复没完全对准隐藏测试”。

严重度在这里按三件事估计：失败测试数量，涉及模块是否是核心路径，以及运行轨迹中是否已经出现 loop 或 budget 信号。django__django-11019、django__django-13321、django__django-15252 属于高严重度；前两者目标失败面很大，后一者涉及测试数据库创建和迁移表行为，并伴随预算耗尽。其余大多是中低严重度的局部语义偏差。

实例	F2P 失败	严重度	风险信号	错误原因摘要
astropy__astropy-14365	1	低	无	qdp.py 只加了命令解析大小写无关匹配，但 test_roundtrip[True] 仍失败，说明 roundtrip 写回/读回语义没有完整覆盖。
astropy__astropy-7746	1	中	无	wcs.py 对零长度输入做了短路，但 test_zero_size_input 仍失败，说明返回形状或不同调用形式的空数组语义仍不符合 WCS 测试契约。
django__django-11019	13	高	loop + budget	widgets.py 大改 media 合并算法后仍有 13 个 forms/admin media 测试失败；这是核心算法语义未收敛，且运行轨迹有 file_read loop 和 lead budget_exceeded。
django__django-11283	1	中	无	auth proxy permission 迁移仍挂已有目标 permission 的场景；patch 删除/复用冲突权限的策略没有完全匹配迁移测试对数据保留和更新顺序的要求。
django__django-11564	2	低	loop	FileSystemStorage.url() 加了 SCRIPT_NAME 前缀，但 test_add_script_name_prefix 和 test_not_prefixed 仍失败，说明绝对/相对 URL 前缀判断边界不准。
django__django-11630	2	中	loop	duplicate db_table 检查在有 DATABASE_ROUTERS 时简单降为 warning，但路由器可分库场景和同库冲突场景没有区分清楚。
django__django-11905	2	中	无	IsNull.as_sql() 加了布尔类型硬检查，但 test_isnull_non_boolean_value 和 iterator 场景仍不符，说明异常类型、触发时机或表达式 RHS 处理仍偏离预期。
django__django-13321	18	高	无	session legacy decode 只把 base64.b64decode() 放进 try，但 18 个 session backend 的 legacy decode 和安全日志用例仍失败，核心会话兼容路径没有真正修稳。
django__django-13768	1	中	无	send_robust() 加了 logger.exception()，但 test_send_robust_fail 仍失败，通常意味着 logger 名称、消息模板或异常记录对象不符合测试断言。
django__django-14155	3	中	budget	ResolverMatch partial 处理同时改了 RoutePattern.__str__(), 但 repr 和 request 相关测试仍失败；当前 patch 对 partial unwrap、args/kwargs 合并和 route 字符串化的组合语义仍不一致。
django__django-14730	1	中	loop + budget	ManyToManyField 新增对 symmetrical related_name 的硬错误，但官方仍挂 useless related name 测试；判断条件或错误等级不符合 Django 既有 check 体系。
django__django-15252	2	高	budget	MigrationRecorder 在 allow_migrate() 为 false 时早退，但测试数据库创建和 django_migrations 表不存在场景仍失败；这是迁移记录器与测试数据库初始化的核心交互问题。

实例	F2P 失败	严重度	风险信号	错误原因摘要
django__django-15695	1	中	budget	<code>RenameIndex.database_backwards()</code> 尝试恢复无名索引自动名, 但 <code>test_rename_index_unnamed_index</code> 仍失败, 说明反向重命名的状态选择、索引定位或名称生成仍有偏差。

表 14: 更新仓库补测后仍未通过的 13 个实例

从分布看, 13 题里没有新的 patch apply 问题, 也没有 P2P 回归, 说明更新后的 Harness 至少把产物稳定性维持住了; 真正剩下的是更细的语义对齐。高严重度样本集中在 media 合并、session decode、迁移记录器这些跨模块或核心生命周期路径; 中严重度样本多是单个边界测试尚未命中; 低严重度样本则更像局部格式、前缀或 roundtrip 行为没打透。风险信号层面, 11019、11564、11630、14730 出现过 loop 信号, 11019、14155、14730、15252、15695 出现过 `budget\_exceeded`。这说明 loop/budget 仍值得监控, 但这批失败的主因仍然是 patch 语义不足, 工具层崩溃的解释力较弱。

16 外部方法在 SWE-bench Lite 上的公开表现

16.1 官网数据

为了给本地结果找参照, 我们从 SWE-bench 官网截至 2026-06-29 可见的 `leaderboard-data` JSON 中抽取了 Lite 条目, 并结合 SWE-bench Lite 官方页面的说明整理公开数字。数据源是 SWE-bench 官网首页 <https://www.swebench.com/> 和 Lite 页面 <https://www.swebench.com/lite.html>。SWE-bench Lite 一共 300 题, 官网每个条目的 `\% Resolved` 都按 300 题口径计算。本报告的 OpenCollab + GLM-5.2 结果只覆盖 Lite 前 100 题, 因此不能直接当作完整 Lite 榜单成绩; 它只能说明这 100 个实例上的局部表现。官网首页已经有 2026 年的 Verified、bash-only、Multilingual 等榜单记录, 但 Lite 分组当前可见的最新日期是 2025-09-11, 其他分组的新日期不能直接并入 Lite 口径。下表列出官网 Lite 分组当前可见的全部 84 条记录, 按日期倒序排列; 同名方法的多次提交保留为多行, 因为官网把它们作为独立条目录录。

方法	日期	Lite 通过率
Isea	2025-09-11	51.33%
KGCompass + Claude 4 Sonnet (20250514)	2025-09-06	58.33%
EntroPO + R2E + Qwen3-Coder-30B-A3B-Instruct	2025-09-01	49.67%
EntroPO + R2E + Qwen3-Coder-30B-A3B-Instruct	2025-09-01	45.00%
MCTS-Refine-7B	2025-06-27	16.33%
ExpeRepair-v1.0 + Claude 4 Sonnet	2025-06-25	60.33%
SemAgent_Multi-v1.0	2025-06-25	51.67%
KGCompass + Claude 3.5 Sonnet (20241022)	2025-06-19	46.00%
ExpeRepair-v1.0	2025-06-13	48.33%
KGCompass + DeepSeek V3	2025-06-09	36.67%

方法	日期	Lite 通过率
Codev	2025-05-28	49.00%
SWE-agent + Claude 4 Sonnet	2025-05-26	56.67%
Codart AI	2025-05-15	41.67%
Lingxi	2025-05-09	42.67%
Refact.ai Agent	2025-04-25	60.00%
CodeFuse-CGM	2025-03-10	44.00%
SWE-Fixer (Qwen2.5-7b retriever + Qwen2.5-72b editor)	2025-03-06	24.67%
SWE-agent + Claude 3.7 Sonnet	2025-02-26	48.00%
Agentless Lite + O3 Mini (20250214)	2025-02-14	32.33%
Aegis - o3-mini_1.0	2025-02-07	30.33%
DARS Agent	2025-02-05	47.00%
Isoform	2025-01-14	55.00%
Moatless Tools + Claude 3.5 Sonnet (20241022)	2025-01-14	39.00%
OrcaLoca + Agentless-1.5 + Claude-3.5 Sonnet (20241022)	2025-01-13	41.00%
OpenCSG Starship Agentic Coder + GPT 4 (0806)	2025-01-13	39.67%
Moatless Tools + Deepseek V3	2025-01-11	30.67%
Patched.Codes Patchwork	2025-01-04	37.00%
CodeFuse-AAIS	2025-01-04	35.67%
Blackbox AI Agent	2024-12-20	49.00%
PatchKitty-0.9 + Claude-3.5 Sonnet (20241022)	2024-12-20	41.33%
Gru(2024-12-08)	2024-12-08	48.67%
Kodu-v1 + Claude-3.5 Sonnet (20241022)	2024-12-07	44.67%
Kortix AI (claude-3-5-sonnet-20241022)	2024-12-03	30.00%
Agentless-1.5 + Claude-3.5 Sonnet (20241022)	2024-12-02	40.67%
SWE-Fixer (Qwen2.5-7b retriever + Qwen2.5-72b editor) 20241128	2024-11-28	23.33%
Globant Code Fixer Agent	2024-11-27	48.33%
devlo	2024-11-22	47.33%
Moatless Tools + Claude 3.5 Sonnet (20241022)	2024-11-17	38.33%
reproducedRG	2024-11-17	28.00%
AppMap Navie v2	2024-11-13	36.00%
CodeShellTester + GPT 4o (2024-05-13)	2024-11-11	31.33%
Composio SWE-Kit (2024-10-30)	2024-10-30	41.00%
Agentless-1.5 + GPT 4o (2024-05-13)	2024-10-28	32.00%
OpenHands + CodeAct v2.1 (claude-3-5-sonnet-20241022)	2024-10-25	41.67%
IBM AI Agent SWE-1.0 (with open LLMs)	2024-10-16	23.67%
HyperAgent	2024-09-25	25.33%
Bytedance MarsCode Agent	2024-09-12	39.33%
AIGCode Infant-Coder(2024-08-30)	2024-09-08	30.00%
Isoform	2024-08-29	35.00%
Bytedance AutoSE (based on SWE-Agent) + GPT4/GPT4o Mixed (20240828)	2024-08-28	21.67%
Honeycomb	2024-08-20	38.33%
Gru(2024-08-11)	2024-08-11	35.67%
Agentless + RepoGraph + GPT-4o	2024-08-08	29.67%
SuperCoder2.0	2024-08-06	34.00%
SWE-agent + GPT 4o (2024-05-13)	2024-07-28	18.33%
OpenHands + CodeAct v1.8	2024-07-25	26.67%
Bytedance MarsCode Agent + GPT 4o (2024-05-13)	2024-07-23	34.00%

方法	日期	Lite 通过率
Amazon Q Developer Agent (v20240719-dev)	2024-07-21	29.67%
SIMA + GPT 4o (2024-05-13)	2024-07-06	27.67%
CodeStory Aide + Mixed Models	2024-07-02	43.00%
Agentless + GPT 4o (2024-05-13)	2024-06-30	27.33%
AbanteAI MentatBot + GPT 4o (2024-05-13)	2024-06-27	38.00%
Moatless Tools + Claude 3.5 Sonnet	2024-06-23	26.67%
Alibaba Lingma Agent	2024-06-22	33.00%
AutoCodeRover (v20240620) + GPT 4o (2024-05-13)	2024-06-21	30.67%
SWE-agent + Claude 3.5 Sonnet	2024-06-20	23.00%
Factory Code Droid	2024-06-17	31.33%
Moatless Tools + GPT 4o (2024-05-13)	2024-06-17	24.67%
AppMap Navie + GPT 4o (2024-05-13)	2024-06-15	21.67%
MASAI + GPT 4o (2024-05-13)	2024-06-12	27.33%
IBM Research Agent-101	2024-06-12	26.67%
CodeR + GPT 4 (1106)	2024-06-04	28.33%
AutoCodeRover (v20240408) + GPT 4 (0125)	2024-05-30	19.00%
OpenCSG StarShip CodeGenAgent + GPT 4 (0613)	2024-05-24	23.67%
Aider + GPT 4o & Claude 3 Opus	2024-05-23	26.33%
Amazon Q Developer Agent (v20240430-dev)	2024-05-09	20.33%
SWE-agent + GPT 4 (1106)	2024-04-02	18.00%
SWE-agent + Claude 3 Opus	2024-04-02	11.67%
RAG + Claude 3 Opus	2024-04-02	4.33%
RAG + GPT 4 (1106)	2024-04-02	2.67%
RAG + Claude 2	2023-10-10	3.00%
RAG + SWE-Llama 7B	2023-10-10	1.33%
RAG + SWE-Llama 13B	2023-10-10	1.00%
RAG + ChatGPT 3.5	2023-10-10	0.33%

表 15: SWE-bench 官网 Lite 组当前可见全部条目

16.2 官网外近期论文和技术报告

只看官网会漏掉 2025 年 9 月之后的很多实验。我们额外搜索了 2025-10 至 2026-06 期间包含 **SWE-bench Lite**、**SWE-Lite**、**Pass@1**、**resolve rate** 等关键词的 arXiv 论文和技术报告，并下载 PDF 从正文表格抽取数字。下表只放作者明确报告为标准 SWE-bench Lite 300 题的结果；它们多数尚未出现在官网 Lite checked 榜单里，所以可复查性弱于官网，但能反映近期研究已经推进到的实验范围。来源包括 HGM <https://arxiv.org/abs/2510.21614>、ConRAD <https://arxiv.org/abs/2601.23257>、CodePilot <https://arxiv.org/abs/2602.00129>、RepoRepair <https://arxiv.org/abs/2603.01048>、SWE-Adept <https://arxiv.org/abs/2603.01327>、Agent-CoEvo <https://arxiv.org/abs/2604.04580>、AgentForge <https://arxiv.org/abs/2604.13120> 和 TACO <https://arxiv.org/abs/2604.19572>。

系统/论文	日期	模型/设置	Lite 结果	口径说明
HGM Best-belief SWE-Verified Agent	2025-10-24	GPT-5	57.00%	标准 300 题；论文称与当时官网最佳 checked 人工设计 agent 持平。
HGM Best-belief SWE-Verified Agent	2025-10-24	GPT-5-mini	49.00%	标准 300 题；由 SWE-Verified 上搜索得到的 agent 转测 Lite。
SWE-agent baseline in HGM	2025-10-24	GPT-5-mini	47.60%	HGM 论文内部复现的对照组，用于隔离 agent 设计收益。
ConRAD	2026-01-30	GPT-5	40.00% (120/300)	Agentless scaffold 上加入 outcome-conditioned repair plan。
ConRAD	2026-01-30	GPT-4o	37.70% (113/300)	相比 Agentless GPT-4o 的 27.30% 提升 10.4 个点。
ConRAD	2026-01-30	DeepSeek-V3	35.60% (107/300)	相比 Agentless DeepSeek-V3 的 27.00% 提升 8.6 个点。
CodePilot	2026-01-28	Qwen3-8B	24.67%	开权重模型；MCTS、thinking mode 和执行反馈结合。
RepoRepair	2026-03-01	DeepSeek-V3 + Claude-4	45.70% (137/300)	文档增强定位和修复；论文报告平均 \$0.44/题。
SWE-Adept	2026-03-01	Claude-4.5	71.30%	两 agent 深度定位加结构化修复；论文报告约 3119k tokens/题。
SWE-Adept	2026-03-01	GPT-5.2	58.80%	同一框架；论文报告约 703k tokens/题。
Agent-CoEvo	2026-04-06	DeepSeek-V3	41.33% (124/300)	代码 patch 与测试 patch 共同演化；论文报告平均 \$1.11/题。
AgentForge	2026-04-13	GPT-4o	40.00%	多 agent + 沙箱执行验证；固定 retry 预算，论文估计全 300 题约 \$43.35。
TACO + terminal agent	2026-04-21	MiniMax-M2.5	57.12%	观测压缩框架；baseline 为 56.30%，总 token 从 307.61M 降到 270.53M。

表 16: 官网外近期论文中标准 SWE-bench Lite 300 题结果

这张表说明，官网 Lite 榜单停在 2025-09 并不代表之后没有相关结果。2026 年的论文里已经出现了若干新模型、新 scaffold 和新推理策略的 Lite 评测，其中 SWE-Adept + Claude-4.5 报告了 71.30%，明显高于官网当前可见最高的 60.33%。这个数字需要谨慎读：论文内部实验、官方 checked 提交、我们本地多轮重跑的前 100 题，三者的可复查程度和提交协议不同。严肃比较时，官网 checked 结果仍然是最稳的参照；判断研究趋势时，官网外论文必须纳入，否则会系统性低估 2025 年 9 月之后的进展。

围绕 GPT-5.5 和 Claude 4.6，我们又单独查了 SWE-bench 官网 JSON、Anthropic system card、OpenAI 发布页、CodeClash 官网和 Claw-SWE-Bench 论文。当前能确认的情况是：标准 SWE-bench Lite 300 题公开榜单里没有这两个模型的直接条目；Claude 4.6 的公开 SWE-bench 数字主要落在 Verified 和 Multilingual 口径；GPT-5.5 的公开 coding benchmark 数字主要来自 OpenAI 发布页的 SWE-Bench Pro、Terminal-Bench 2.0，以及 Claw-SWE-Bench 的 full-350 模型 sweep。CodeClash 官网截至 2025-11-03 的 leaderboard 只列到 Claude Sonnet 4.5、GPT-5、o3 等 8 个模型，没有 GPT-5.5 或 Claude 4.6；独立名为 CBench 或 C-Bench 的可引用公开成绩也未定位到。

来源	日期	benchmark 口径	模型结果	解释
SWE-bench 官网 JSON	2026-02-17	SWE-bench Verified, 500 题	mini-SWE-agent + Claude Opus 4.6: 75.6%	官网 leaderboard-data 的 Verified 条目；同一条目总成本 \$275.76，平均 \$0.55/题。
SWE-bench 官网 JSON	2026-02-13	SWE-bench Multilingual, 300 题	Claude 4.6 Opus: 72.0%	官网 Multilingual 条目；同一条目总成本 \$198.89，平均 \$0.66/题。
Anthropic Sonnet 4.6 system card	2026-02-17	SWE-bench Verified / Multilingual	Sonnet 4.6: Verified 79.6%, Multilingual 75.9%; Opus 4.6: Verified 80.8%; GPT-5.2: Verified 80.0%	厂商 system card 的横向表；脚注说明 Sonnet 分数为 10 次平均，prompt modification 后到 80.2%。
Anthropic Opus 4.6 发布页	2026-02-05	SWE-bench Verified	Opus 4.6: 81.42%	发布页脚注说明该值来自 prompt modification；常规表中 Opus 4.6 为 80.8%，25 次平均。
OpenAI GPT-5.5 发布页	2026-04-23	SWE-Bench Pro / Terminal-Bench 2.0	SWE-Bench Pro: 58.6%; Terminal-Bench 2.0: 82.7%	官方发布页没有给标准 SWE-bench Lite 或 Verified 数字；它报告的是更难的 SWE-Bench Pro 和命令行任务口径。
Claw-SWE-Bench	2026-06-10	full-350 派生 benchmark	GPT 5.5: 78.0% (273/350); Claude Opus 4.7: 77.1% (270/350)	固定 OpenClaw scaffold 的九模型 sweep；未报告 Claude 4.6 正式成绩。GPT 5.5 全量成本 \$1399.1。
CodeClash 官网	2025-11-03	6 个 arena 的 goal-oriented software engineering ELO	Claude Sonnet 4.5: 1385; GPT-5: 1366	官网当前榜单没有 GPT-5.5 或 Claude 4.6；该 benchmark 按竞技场胜负给 ELO，与 SWE-bench resolved 率不同。
CBench / C-Bench 检索	2026-06-29	未确认到稳定公开口径	未定位到 GPT-5.5 或 Claude 4.6 成绩	公开搜索未找到可引用的原始榜单或论文结果；后续比较不把它计入数值表。

表 17: GPT-5.5 和 Claude 4.6 相关公开 coding benchmark 结果

还有一批近期工作使用了 SWE-bench Lite 的子集、相关任务或派生 benchmark。这些数字不能折算成 300 题榜单成绩，但它们对理解“为什么同一模型在不同 harness 下差这么多”很有用。MarketBench <https://arxiv.org/abs/2604.23897> 在 93 题子集上比较了六个新模型的自我校准；Phoenix <https://arxiv.org/abs/2606.20243> 在 24 题 curated slice 上报告生产 webhook 路径表现；SWE-ContextBench <https://arxiv.org/abs/2602.08316> 从 SWE-bench Lite 构造了 300 个 base tasks 和 99 个 related tasks；Claw-SWE-Bench <https://arxiv.org/abs/2606.12344> 发布了另一个 80 题 Lite-80，来源为 SWE-bench Multilingual 和 Verified-Mini，口径不同于 SWE-bench Lite 300 题本身。

来源	日期	子集/派生口径	主要数字和解释
MarketBench	2026-04-26	93-task SWE-bench Lite subset; 另有 50-task common slice	93 题上 Claude Opus 4.5、Gemini 3 Pro Preview、GPT-5.2 均为 80.6%; GPT-5.2-pro 为 79.6%, Claude Sonnet 4.5 为 77.4%, GPT-5-mini 为 75.3%。50 题 common slice 上, external scaffold 的 Solo GPT-5.2 为 37/50, Codex + GPT-5.2 1800s 诊断为 35/50。
Phoenix	2026-06-18	24-instance curated SWE-bench Lite slice	18/24, 即 75% oracle-resolved; 作者明确提醒该 slice 与完整 leaderboard 不可直接比较。
SWE-ContextBench Lite	2026-02-09	300 base tasks + 99 related tasks; 主要报告 99 related tasks	Claude Sonnet 4.5 的 no-context baseline 为 26.26%, oracle summary learning 为 34.34%; Supermemory 在相关任务上为 30.30%。该实验评测上下文复用, 口径不同于标准 Lite 300 题。
Claw-SWE-Bench Lite-80	2026-06-10	80-instance 另建 benchmark, 来源为 Multilingual + Verified-Mini	完整 350 题中 GPT 5.5 为 78.0% (273/350), Claude Opus 4.7 为 77.1% (270/350), GLM 5.1 为 73.4% (257/350); Lite-80 被设计为以约 23% 成本近似 full-350, 来源与 SWE-bench Lite 300 题不同。

表 18: 近期子集或派生口径结果, 不能直接折算为 SWE-bench Lite 300 题榜单成绩

17 Loop detect 的全局作用评估

为了回答“全量 GLM-5.2 实验里是否有必要监测重复行为, Loop detect 到底有没有发挥关键作用”, 我们重新扫描了 eval_work 下所有 *.events.jsonl。当前可见事件日志共 87 个, 其中 12 个文件出现过真实的 loop_detected, 总计 56 次; 13 个文件出现 error 事件, 其中 4 个同时有 loop 事件。这个比例说明重复工具调用虽然没有构成主流失败形态, 但已经超过了可以忽略的噪声水平。

OpenCollab 的 loop detector 工作在工具调用层。普通工具的完全相同参数重复 3 次就会拦截; file_read 会按路径归一化, 允许同一文件被读到第 8 次才拦截; 最近调用窗口是 200。触发后, 系统会把 loop_blocked 写入 trace, 向模型返回一条 You are stuck in a loop 的工具消息, 并发出 loop_detected 事件。它会阻止那一次重复工具真正执行, 但不会自动终止整个实例。

统计项	数值	含义
事件日志总数	87	本报告阶段可扫描的 OpenCollab 运行日志
含 loop_detected 的日志	12	至少一次重复工具调用被拦截
loop_detected 总次数	56	全部被记录的重复工具拦截事件
含 error 的日志	13	运行期出现错误事件
同时含 error 与 loop 的日志	4	环境/工具错误和重复行为相互放大的样本

最典型的三类证据如下。django__django-13590 在 temperature=0.0 的 22 题实验中出现 9 次 loop 事件, bash 重复计数最高到 10, 但后续仍跳出了局部循环, 生成 patch, 并且官方评测通过。这说明 loop detector 有时能当作有效的方向盘, 把会话从重复工具尝试里推出来。django__django-15738 则是反例: 同一轮 temperature=0.0 实验里, 它出现 27 次 loop 事件, file_read 最高到 11, bash

最高到 18，随后触发 `budget_exceeded` 并没有产出可用修复。这里 loop detector 记录了灾情，也拦住了重复工具，但没有强到足以中断烧 token。`django__django-15213` 的 1M 预算复跑显示了另一种盲区：自然语言判断重复 5 次，却没有触发工具层 loop，因为它没有形成足够严格的同工具同参数重复。

因此，在这批 GLM-5.2 实验里，Loop detect 的结论可以说得比较稳：通过率主要由语义修复质量决定；大多数没过的题仍然是语义修复差边界、漏改配套路径或写错实现。可是它确实是全量评测必须保留的护栏。没有它，大部分正常题可能照样能跑完，但低温、工具访问失败、资源异常、读文件原地打转这些场景会更容易把预算烧在重复动作上。尤其在 300 题级别运行时，少数 15738 这种样本足以造成明显 token 浪费和难排查的失败记录。

后续更合理的做法，是保留现有工具层 loop detector，同时再加一层实例级监控。当单个实例的 `loop_detected` 超过 5 次，应在 summary 里标红；超过 10 次或同一句自然语言判断重复超过 3 次，应自动保存最近一次成功写入、当前 `git diff`、最近三次工具错误、最近三段 assistant 文本，并建议停止当前验证动作，改由另一个角色复核。这样不会误伤偶尔重读文件的正常行为，也能避免 `django__django-15738` 那种“相信编辑已经成功、反复证明同一件事”的预算消耗。